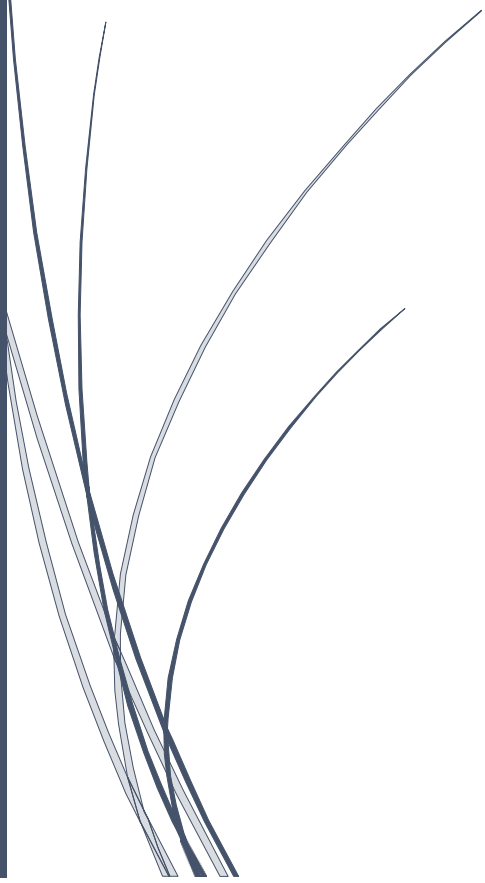
28/03/2025

Invoice-Based Dimensional Modelling Project



Chidinma Ukandu
HNG STAGE 8

SECTION 1: Data Profiling & Inference

Before designing the model, I explored the sample invoice dataset to identify key entities, such as who made purchases, what was bought, when, and where. This business-first approach helped me define four core dimensions, the Customer, Product, Time, and Store where each captures specific attributes essential for sales analysis.

The central table, fact_sales, stores transactional details, linking to each dimension through foreign keys. It includes quantity, unit price, line total, and invoice ID. I set the grain to one row per product per invoice line item, allowing for detailed drill-downs and comparison across products, customers, and time.

Additionally, I handled Invoice_ID as a degenerate dimension which is useful for tracking but not tied to a specific dimension table. This structured foundation supports efficient reporting and future growth.

The Columns Observed

The dataset given contains the following information;

- **Invoice_ID**: A unique code assigned to each customer's transaction.
- **Invoice_Date**: The exact date and time the invoice was created.
- **Customer_ID & Customer_Name**: Who made the purchase.
- **Product_ID & Product_Name**: What item was purchased.
- **Quantity**: How many units were bought.
- **Unit_Price & Line_Total**: The cost per item and the total paid.
- **Store_ID**: Which store the purchase happened in.

At this stage, my goal is to analyze the data with care, not just from a technical point of view but from a business perspective.

Identifying the Dimensions (The "Who, What, When, Where")

From the business point of view, it is known that most times, stakeholders often care about the mentioned below;

- **Who** bought something? → Customers
- **What** was bought? → Products
- **When** was it bought? → Time
- **Where** was it bought? → Stores

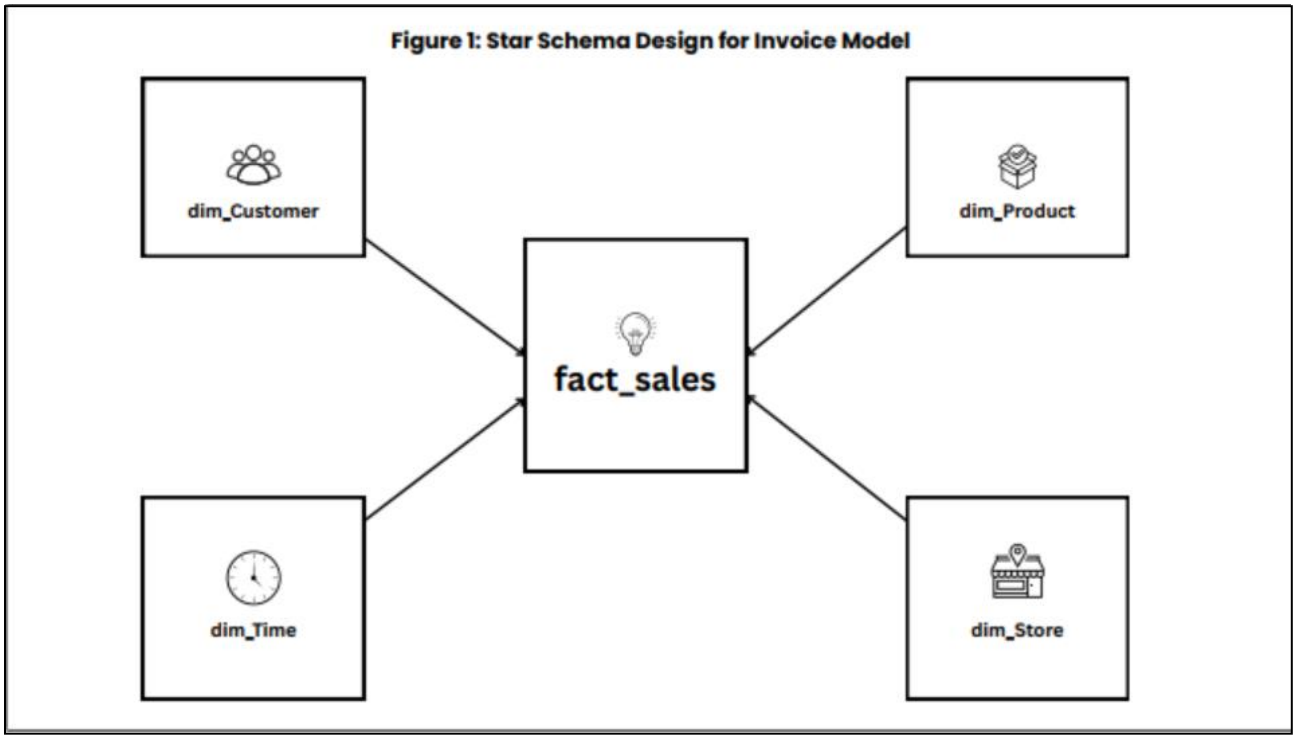
I used these descriptive entities to form my dimension tables as shown below.

Table	Description
dim_customer	Stores details about each customer
dim_product	Lists the products and their prices
dim_time	Breaks down invoice dates into day, month, year, etc.
dim_store	Identifies the location or branch where the sale happened

Table 1

Table 1 has been structured to help workers and stakeholders to easily slice and dice the data.

Defining the Fact Table to capture the "What Happened?"



This diagram shows the central fact_sales table connected to its supporting dimension tables which are; dim_customer, dim_product, dim_store, and dim_time, enabling intuitive and scalable analysis.

The fact_sales table serves as the heart of the model, capturing each transaction in its most detailed form, one row per product per invoice line item. It includes foreign keys linking to dimension tables (customer_id, product_id, store_id, time_id), along with measurable values like quantity, unit_price, and line_total.

This level of detail, also called the grain, ensures accurate drill-downs, store comparisons, and product-level insights. Choosing this grain supports both operational and strategic analysis.

A special case within the fact table is the inclusion of Invoice_ID, a degenerate dimension. While it doesn't belong to a dimension table, it's crucial for tracking and grouping transactions. By keeping it in the fact table, the model maintains invoice-level clarity without unnecessary complexity.

This thoughtful structure balances flexibility, precision, and real-world business needs, forming a solid base for deeper analytics.

SECTION 2: Dimensional Modeling Strategy

I structured the model using a star schema, placing fact_sales at the center and connecting it to the four-dimension tables. This layout simplifies querying and supports intuitive, multi-level analysis.

The schema enables drill-downs from total sales to more specific views by product, time, customer, or store. It also ensures strong performance due to fewer joins and is easily scalable as more data is added.

Real-world needs like analyzing individual purchases or introducing new dimensions in the future are supported by the model's flexible structure. The star schema not only improves data clarity and speed but aligns perfectly with business goals.

At the center of the design is the **fact_sales** table, surrounded by four key dimension tables:

Dimension Table	Represents...	Why it's important
dim_customer	The buyer	Helps analyze buying behavior and segment customers
dim_product	The item sold	Supports tracking product performance
dim_time	Date & time	Enables trend analysis, seasonality tracking
dim_store	The location	Allows store-level comparison and regional insights

Table 2

Relevance of the Star Schema to invoice based dimensional modelling project

The star schema isn't just for creating neat visuals, it has a very powerful functionality. Here's why it's the right choice for this project:

1. Simplicity for Users

The design is intuitive for non-technical users. Analysts can quickly understand how data is structured and query it with ease.

2. Supports Drill-Down and Slice-and-Dice

With the schema in place, users can:

- Drill down from total sales to store-level sales
- Compare product categories by customer segments
- Analyze trends over specific time periods

3. Performance-Oriented

Because the star schema reduces the number of joins compared to other models (like snowflake schemas), it provides faster query performance, especially when the fact table grows over time.

4. Scalable Design

As the business expands (more customers, more stores, more products), this model can easily scale by adding rows to the fact table or extending dimensions without major structural changes.

SECTION 3: Handling Slowly Changing Dimensions (SCD)

In real-world data, some values change over time, like customer names or product prices. To manage these, I applied Slowly Changing Dimensions (SCD) strategies.

I considered three types:

- **Type 1** (overwrite): For minor updates that don’t require history.
- **Type 2** (track full history): For important changes like store location shifts.
- **Type 3** (limited history): For tracking just the current and previous value.

For this project:

- dim_customer and dim_store use Type 2 for historical tracking.
- dim_product can use Type 1 or 2 depending on business need.
- dim_time remains unchanged.

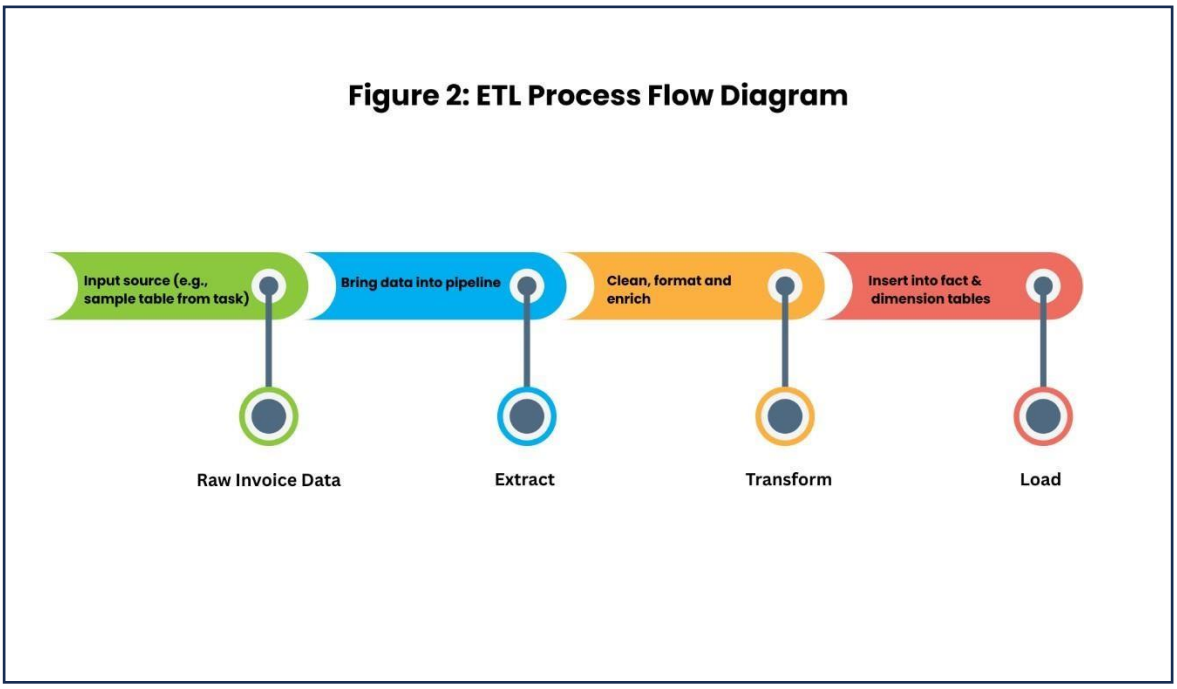
Using a mix of Type 1 and Type 2 as shown in table 3 to ensure data integrity while supporting flexible, meaningful insights over time.

Dimension Table	SCD Strategy	Why
dim_customer	Type 2	Customer info (e.g., address changes) needs to be tracked over time
dim_product	Type 1 or Type 2	Price may change—Type 2 for accuracy, Type 1 for simplicity
dim_store	Type 2	Store relocations or renaming may affect regional analysis
dim_time	No actions	Time doesn't change, so no SCD needed here

Table 3

SECTION 4: ETL Strategy: Preparing Data for Analysis

To power the dimensional model, I designed a structured **ETL (Extract, Transform, Load)** process that ensures clean, accurate, and scalable data flow. The end-to-end data pipeline is illustrated in Figure 2 below.



Step 1: Extract

The process begins with the sample invoice dataset provided in the task instructions. In a real-world setting, this kind of data would typically be extracted from a CSV file, database, or ERP system.

- **Source**: sample_invoice_data.csv
- **Method**: Manual upload, script, or API
- **Goal**: Bring raw invoice records into the ETL pipeline

Step 2: Transform

Here, the data is cleaned and structured to match the star schema format. The key targets include:

- Splitting data into dimension-specific fields
- Removing duplicates
- Creating surrogate keys
- Formatting dates for dim_time
- Calculating line_total
- Applying SCD logic to track changes

Tools that can be used are SQL, Python (Pandas), Power Query, or standard ETL platforms.

Step 3: Load

Once transformed, the data is loaded into the appropriate tables:

- fact_sales
- dim_customer
- dim_product
- dim_store
- dim_time

This ensures a clean, query-ready dataset for analysis and dashboards.

To improve efficiency, future updates should be incremental, loading only new or changed records based on timestamps or keys. This saves time and avoids duplication.

Data Quality & Consistency

Here's how I ensure reliability across all stages:

Quality Check	Description
Uniqueness	Prevents duplicate invoice or product entries
Null checks	Cleans or replaces missing values in key fields
Referential integrity	Verifies foreign keys match existing records in dimension tables
SCD handling	Preserves historical accuracy during updates

These checks can be automated using validation scripts or data quality tools.

This pipeline supports both daily reporting and long-term trend analysis, with performance and trust at its core.

SECTION 5: Advanced Analysis & Scalability

To ensure long-term value, this dimensional model was designed with scalability and advanced analytics in mind. As the dataset grows, either through more transactions, customers, or stores, the star schema remains efficient, readable, and easy to extend.

Hierarchical Relationships & Drill-Down

Each dimension can be expanded to support deeper analysis:

- **Time hierarchy** (Day → Month → Year) allows trend exploration over time.
- **Store regions** or **product categories** can be added to support grouped analysis.
- This structure enables decision-makers to drill down from total sales to specifics, such as sales by customer type, location, or time period.

Performance Optimization Strategies

To maintain performance as data scales:

- **Surrogate keys** are used for efficient joins.
- **Indexes** can be applied to frequently queried fields (like product_id or invoice_date).
- **Partitioning** large fact tables by time can improve query speed.
- **Incremental ETL** ensures only new/changed data is processed during updates.

Flexibility for Future Enhancements

As business needs evolve, the model can accommodate:

- New dimensions (e.g., promotions, payment methods)
- Additional fact tables (e.g., returns, inventory)
- Integration with machine learning tools for forecasting or segmentation

This model not only solves today's reporting needs but is built to support future data growth, complex queries, and evolving business goals—making it a robust foundation for a modern analytics platform.

Section 6: Assumptions & Trade-Offs

Assumptions

To build this model, I made a few simple assumptions:

- The sample invoice data represents typical sales activity.
- Each line on the invoice is unique and can be used to track individual purchases.
- Customer, store, and product info stay the same within one transaction.
- Time details (like date and hour) are accurate and usable.
- Any future changes to customer or store data can be tracked using versioning (SCD).

Trade-Offs

I also had to make a few decisions to balance simplicity and performance:

- Using Type 2 SCD (to track history) gives better insights but makes the system a bit more complex.
- A star schema keeps the model easy to read and fast to query, even though it repeats some data.
- Keeping Invoice_ID inside the fact table (instead of moving it elsewhere) makes reporting easier.
- Using incremental data loading saves time but requires extra checks to avoid missing changes.

These choices were made to keep the model flexible, efficient, and practical for real analysis.

Conclusion

This project transformed a simple invoice dataset into a structured and scalable dimensional model. Through clear profiling, thoughtful schema design, and a well-defined ETL strategy, I created a solution that supports deep analysis, future data growth, and real-world reporting needs. The model not only meets today's goals but also provides a solid foundation for smarter decision-making tomorrow.